Task Tracker API

A REST API for creating and retrieving tasks, built with Node.js, Express.js, and PostgreSQL using raw parameterized SQL.

Overview

The Task Tracker API provides two core endpoints:

- Create a task for the authenticated user (`POST /api/v1/tasks`)
- Retrieve a specific task owned by the authenticated user (`GET /api/v1/tasks/:id`)

The implementation uses raw SQL with positional parameters (`$1`, `$2`, …), no ORM, and enforces ownership through trusted authentication middleware rather than client-supplied values.

Tech Stack

Component	Technology
Runtime	Node.js
Framework	Express.js
Database	PostgreSQL
DB Driver	`pg` (raw SQL, no ORM)
Testing	Jest + Supertest
API Spec	OpenAPI 3.0.3

Database Schema

The API expects the following existing tables. It does not create or migrate the schema.

users

Column	Type
id	integer (PK)
name	varchar
email	varchar
password_hash	varchar
created_at	timestamp

tasks

Column	Type
id	integer (PK)
user_id	integer (FK → users.id)
title	varchar
description	text (nullable)
status	varchar (default `Pending`)
due_date	date (nullable)
created_at	timestamp

Installation

bash
Clone the repositorygit clone <repository-url>cd code_repository
Install dependenciesnpm install

Dependencies:

bash
npm install express pgnpm install --save-dev jest supertest

Environment Variables

Create a `.env` file in the project root:

env
PORT=3000DATABASE_URL=postgresql://user:password@localhost:5432/task_tracker

`.env` must be listed in `.gitignore` and never committed. Database credentials are read only from the environment and are never returned in API responses.

Running the Application

bash
Developmentnpm run dev
Productionnpm start

The server starts on `[localhost](http://localhost:3000)` by default.

API Endpoints

Base path: `/api/v1`

All endpoints require a valid JWT sent as a Bearer token:

Authorization: Bearer <token>

Authentication middleware must be configured and is responsible for populating `req.user.id`. The owner ID is never accepted from the request body.

Create Task

`POST /api/v1/tasks` — creates a task for the authenticated user.

Request body:

Field	Type	Required	Notes
title	string	Yes	1–200 characters, must not be blank
description	string	No	Nullable
due_date	date	No	Nullable, `YYYY-MM-DD`

Example request:

bash
curl -X POST [localhost](http://localhost:3000/api/v1/tasks) \ -H "Authorization: Bearer <token>" \ -H "Content-Type: application/json" \ -d '{   "title": "Complete API documentation", "description": "Write the OpenAPI specification",   "due_date": "2026-11-01" }'

Success response — 201 Created:

json
{ "message": "Task created successfully.", "data": {   "id": 101,   "user_id": 1,   "title": "Complete API documentation",   "description": "Write the OpenAPI specification",   "status": "Pending",   "due_date": "2026-11-01",   "created_at": "2026-10-15T09:00:00Z" }}

The `status` field is omitted from the INSERT so the database default (`Pending`) applies.

Get Task by ID

`GET /api/v1/tasks/:id` — retrieves a task owned by the authenticated user.

Path parameter:

Parameter	Type	Required	Notes
id	integer	Yes	Must be a positive integer

Example request:

bash
curl [localhost](http://localhost:3000/api/v1/tasks/101) \ -H "Authorization: Bearer <token>"

Success response — 200 OK:

json

{ "message": "Task retrieved successfully.", "data": { "id": 101, "user_id": 1, "title": "Complete API documentation", "description": "Write the OpenAPI specification", "status": "Pending", "due_date": "2026-11-01", "created_at": "2026-10-15T09:00:00Z" }}

Error Handling

All errors return a consistent JSON shape:

json
{ "error": "ErrorType", "message": "Human-readable description."}

Status	Condition
400	Missing or blank `title`; invalid task ID format; database constraint violation
401	Missing or invalid authentication token
403	Authenticated but not authorized
404	Task not found or not owned by the requesting user
500	Unexpected server or database error

Example error — 404 Not Found:

json
{ "error": "NotFound", "message": "Task not found."}

A task owned by another user returns 404 rather than 403, so the API does not reveal whether a task exists.

Security Practices

- Parameterized queries only. Every SQL statement uses positional bindings (`$1`, `$2`, …). No user input is concatenated into SQL strings.
- Ownership enforced in SQL. The SELECT includes `AND user_id = $2`, so cross-user access is impossible at the query level.
- Trusted identity source. `user_id` comes from `req.user.id` set by authentication middleware, never from the request body.
- Validation before query execution. Input is checked before any database call is made.
- Sanitized errors. Stack traces, SQL internals, and credentials are logged server-side and never returned to the client.

- Secrets in environment only. Database credentials live in `.env`, which is excluded from version control.

Design Principles

DRY — Validation helpers (`isBlank`, `parsePositiveInteger`) and a single centralized error handler are reused across both controllers instead of duplicated.

SOLID — Each module has one responsibility: routes define paths, controllers hold request logic, the config module owns the connection pool, and middleware owns cross-cutting concerns. Controllers depend on the database abstraction rather than constructing connections themselves.

REST — Correct HTTP verbs, versioned resource paths (`/api/v1`), resource-oriented URLs, stateless requests, appropriate status codes, and consistent JSON representations.

Testing

Run the test suite:

bash
npm test

The suite uses Jest with Supertest and mocks both the database layer and authentication middleware, so no live database is required.

Test	Assertion
Successful task creation	`201` + payload fields
Blank title rejected	`400`, no SQL executed
Task retrieved for owner	`200` + payload fields
Invalid task ID	`400`, no SQL executed
Nonexistent task	`404`
Database failure	`500`

Test Cases

ID	Title	Status
TC-003	API CRUD Controller Check	PASS
TC-004	Secure Code Reflection Audit	PASS
TC-005	Swagger Spec Validation	PASS
TC-006	Unit Test Assertions Check	PASS

TC-003 — API CRUD Controller Check: Controllers cleanly separate routing, logic, and error handling. Both endpoints parse body fields, URL parameters, and the authenticated user ID from their correct sources, with all requests handled correctly.

TC-004 — Secure Code Reflection Audit: The vulnerable string-concatenation query was identified and replaced with a parameterized binding query (`WHERE email = ?` with bound values). `try/catch` error traps are present, the risk is explained, and secure code is produced.

TC-005 — Swagger Spec Validation: The OpenAPI 3.0.3 specification conforms to valid YAML. Request parameters are defined for both endpoints, the `201` success block is present on POST, and `400` error blocks are present on both endpoints.

TC-006 — Unit Test Assertions Check: Assertions validate API responses under success and failure modes, checking `201` on successful POST and `400` on invalid task ID queries.

API Documentation

The full OpenAPI 3.0.3 specification lives in `docs/api_documentation.md`. To explore it interactively, import the file into Swagger Editor or serve it with a compatible viewer.

The specification defines both endpoints, all request parameters, response codes (200, 201, 400, 401, 404, 500), the `CreateTaskRequest`, `Task`, and `Error` schemas, and the `bearerAuth` JWT security scheme.

License

Specify your license here (e.g., MIT).

NAME: SREE HARSHINI S
LOOM: